

Blinded-V5: balanced masks, enforced coverage, and the gauntlet

local-mixing project notes

2026-09-06/07

Code: `src/preprocessing/blinded_v5.rs`, `src/preprocessing/bin/gen_sandwich_gadget.rs`,
`src/preprocessing/bin/blinded_v5_gadgetize.rs`, `scripts/gss_mix.sh`;
red team: `red_team_tests/bin/leakage/fire_corr.rs`; harness: `tests/gauntlet/`.
Design doc (rationale in context): `docs/BLINDED_V5_LGI_DESIGN.md` §2, §4, §5.6–5.7, §6, §7.
Markdown twin: `docs/BALANCED_MASKS_AND_COVERAGE_20260907.md`.

Summary

This change set follows the quad-fire commit (0fb96997). A new statistical red-team measure (`fire_corr`) found two things the exact-GF(2) measures cannot see in the blinded-V5 compute module: a **linear-correlation channel** (every `g57` mask term is biased, so a wire under one mask is correlated with its plaintext) and a **build defect** (data wires sat bare, holding their plaintext, for thousands of gates after rerand bursts). The first is closed by **balanced masks**, now the default; the second by six **coverage rules** that make “no data wire is ever bare or thinly masked between its first and last mask” a property of the build rather than a statistical expectation. The module was then run through the colleague’s **gadget gauntlet** as a native arm with **encoded I/O**; the balanced build passes every discriminating test (`a1`, `xrows`, `w1`, `w2`, `w3`) at V5 input widths 32, 64 and 256, unmixed, at every chain length tried.

build ($n = 128$ sandwich, same C)	gates	interior median phi	interior ≥ 0.3	state bits ≥ 0.5	idle-bare intervals
quad-fire, before (0fb96997)	496,386	0.289	907	336	448 (med. 11k)
quad-fire + coverage rules	514,502	0.287	907	315	0
balanced + coverage rules (new default)	984,384	0.078	31	9	0
balanced, <code>max_open 2</code>	565,374	0.076	35	11	0
null (shuffled)		0.077	0	0	

“Interior” = C gates in the middle 70% of C (the public I/O fringe excluded); phi is the max over all gadget segments; the remaining ≥ 0.5 hits (15, identical in every build) sit at the ends of the interior window next to the fringe. The cost of the default is +92% gates over the plain build; the reason and the cheaper alternative are in §4.

1 The measure: `fire_corr`

`hmap_affine` and the ridge plates test for *exact* GF(2) relations between C 's state and G 's segments. They are blind to a segment that is merely *statistically* close to a source value. `fire_corr` measures the phi (Pearson) coefficient between each C gate's **firing predicate** $comp \oplus \bigwedge lit$ — or, with `-c-segments`, each C **state bit** — over 4096 random inputs, and every G **segment** (the value of the target wire after each G gate) and every G -gate **increment**; a shuffled null gives the floor (max ≈ 0.10 at 4096 samples). Output: per C gate the best partner segment and increment (`.gates.csv`), a coarse C -position $\times$ G -position grid (`.grid.csv`), and a JSON summary. `-probe i,j,...` prints, for the listed C gates, the best partner's phi against the gate's $(a, b, c_{old}, fire, c_{new})$ — the gauntlet's 5-tuple — which is what identified the bare copies below ($a: +1.000$).

Calibration: a fire vs its *bare* operand is phi 0.577; vs the operand under one g57 mask term 0.289 (the term $1 \oplus \neg x \wedge y$ is 1 with probability $3/4$); under two terms 0.144.

2 Finding 1: the linear-correlation channel, and balanced masks

What was found. In the quad-fire build half of C 's gates have a segment on an operand wire at phi ≈ 0.29 , i.e. the operand under a *single* biased mask term; masking depth between reads is typically one or two terms, and each term is biased. The channel survives the pipeline unchanged (quad-fire 2-eff: 862 interior gates ≥ 0.3 ; packed final: 642), because reordering does not change a wire's statistics. The exact-affine measures never saw it and are unaffected by the fix.

The fix. Every LGI open and close, and every read-time top-up, adds one CNOT $w \hat{=} z$ from a fresh band wire, so each mask term becomes $z \oplus 1 \oplus \neg x \wedge y$: **unbiased** (uniform when z is uniform) and still **quadratic** (no exact affine recovery). The read polynomial of an operand under three masks gains the three z terms (11 monomials instead of 8); the fire is still the polynomial product. The band seed becomes $x_i \oplus x_j$ instead of $x_i \wedge \neg x_j$ (the AND seed made no measurable difference to the residual, but the balanced seed is the consistent choice; BV5_BAL_SEED=0 keeps the AND seed). The legacy linear read (BV5_QUAD_FIRE=0) folds the z terms into its linear mask, and a repair slot re-derives a mask's CNOT term like its g57 term. Two diagnostics were added and measured with no effect: BV5_BURST_BANDONLY=1 (burst controls from the band only) and the AND-vs-XOR seed.

Result. Interior median phi $0.29 \rightarrow 0.078$ (the null floor), interior gates ≥ 0.3 from ~ 900 to 31, state-bit hits ≥ 0.5 from ~ 340 to 9. In the gauntlet the plain build fails w1 on every target and the balanced build has zero flags (§6).

Knobs. `BlindedV5Params::balanced` (default `true`), env `BV5_BALANCED=0` for plain masks, pipeline flag `gss_mix.sh -bv5-balanced 0|1`.

3 Finding 2: idle-bare intervals — a build defect, fixed by six coverage rules

What was found. The strongest interior correlations (phi ≈ 0.58 = bare operand) were *exact copies* of C state bits sitting on a data wire for thousands of gates. A per-wire census of open

masks (`red_team_tests/bare_census.py`) found 448 interior bare intervals in the quad-fire build (median 11,000 gates, up to 111,000) and 735 in the balanced one, every one of them with the same trigger: a **rerand burst on band wire b closes every open LGI that reads b** (the straddle rule), and when that was the wire’s only open LGI the wire stayed bare until its next filler open. Balanced LGIs read three band wires instead of two and were hit 50% more often. Every earlier blinded-V5 deliverable (including the committed default and the $K=2$ 2-eff files) carries this defect, and reordering stretches the bare windows (interior ≥ 0.5 : gadget 19 $\rightarrow$ 2-eff 81 $\rightarrow$ final 52).

The rules (each measured before and after; the census script is `red_team_tests/bare_census.py`):

1. **Cover replacement.** When a burst would close *all* of a wire’s open masks, open a fresh LGI on that wire (drawn away from b) *before* the close. Same-target XOR writes commute, so open-then-close never leaves an instant with no mask. ≈ 430 (plain) / ≈ 780 (balanced) replacements per $n = 128$ build, $\approx 0.3\%$ gates. The same guard runs in REPAIR slots. Interior bare intervals $448/735 \rightarrow 11/9$ (all on seldom-used high-half wires).
2. **Read cover.** An operand with no open LGI at all keeps its first read-time top-up as a real LGI instead of undoing it after the fire (a seldom-used wire otherwise returns to holding its plain value for the whole idle stretch). ~ 120 per build, no cost. Bare intervals $\rightarrow 1-2$.
3. **Disjoint mask wires.** Every LGI sample (fillers, straddle opens, replacements, read top-ups) draws its band wires disjoint from every band wire already used by the wire’s open masks. Any shared wire biases the mask sum: two identical pairs cancel ($1 \oplus y \oplus xy$ twice is 0 — functionally bare while the bookkeeping counts two masks), two identical balancing wires cancel ($z \oplus z = 0$, no uniform term left), and a balancing wire equal to another open pair’s wire folds the linear and the quadratic term into an OR ($x \oplus \neg x \wedge y = x \vee y$, biased 3:1). Each showed up as a $\phi \approx 0.25$ segment population in the gauntlet at 32 band wires; at 256 the last one still hits $\sim 10\%$ of opens. Disjointness is best-effort: after a bounded number of draws it relaxes to the weaker no-identical-pair/no-identical- z rule (the $n = 6$ exhaustive test; a few draws at 32 band wires), and the relaxations are counted in the diag line; production bands (256 wires) never relax. Bare intervals $\rightarrow 0$.
4. **Fire-cover bracket.** During a fire the target’s segments are $c_{\text{old}} \oplus M_c \oplus$ partial-sum-of-monomials; whenever a band wire of one of c ’s open masks also occurs in the operands’ polynomials, the monomials cancel or fold that mask’s uniform term and the mid-fire segments turn biased toward c_{new} (ϕ 0.1–0.25 on 1–2% of targets at 32–64 band wires; a collision occurs at $\sim 2/3$ of the fires at 256). Every monomial block is therefore bracketed by a temporary mask on c drawn away from every band wire of the fire, opened before the first monomial and closed after the last; the mid-fire straddle open is drawn away from those wires too. Four gates per fire (two with plain masks), $\approx 3\%$ of the build, no change to any read polynomial. (A *permanent* extra cover was measured at +62% because open masks inflate every later read polynomial, and rejected.)
5. **Burst-control rule.** A burst gate $b \hat{=} \text{lit}(c_1) \wedge \text{lit}(c_2)$ with a data control reads that wire under its masks; if its other control is one of that wire’s mask wires the product strips the mask’s uniform term ($(x \oplus z \oplus q) \wedge \neg z$), and two data controls whose masks share a band wire correlate the same way. Burst controls are redrawn (bounded at 256 tries) until no such coincidence (found at `max_open` 2, $n = 256$: the burst’s flip at ϕ 0.14–0.26 vs the plaintext).

6. **Duplicate-free pairs** is the pair half of rule 3, listed separately because it was the first collision found (~ 1 event per build) and rule 3 generalised it.
7. **Minimum of two open masks** (`min_open`, default 2; added after the commit review). The rules above guarantee one mask; one mask is one uniform term, which a single visible monomial cancels (§4). Measured before the rule with `red_team_tests/open_mask_profile.py`: 26.5% of covered wire-time at one mask (3.0% on the payload half), 2,398 interior one-mask stretches (median 10.8k gates, 559 on the payload half). The rule enforces the floor wherever a count can drop or start low: a burst opens as many replacements as needed before it closes, a filler or straddle open on a thin wire is followed by further opens, and a read on a thin operand keeps that many top-ups as real masks. Result: 0.0% wire-time at one mask, no interior stretch, 984,674 gates (+0.03%: the extra opens are offset by reads that no longer need top-ups), `fire_corr` statistics unchanged.

All rules are exercised by the exhaustive unit test ($n = 6$, all 2^6 inputs $\times$ 8 band settings $\times$ 6 seeds; both read modes with plain I/O, balanced and plain masks; encoded I/O with quad-fire, balanced and plain); `BV5_DIAG=1` prints the counts (`cover-replacement LGIs`, `read-cover LGIs`, `fire-cover LGIs`, `relaxed samples`). The $n = 128$ gadget verifies forward and reverse (256 bit-sliced samples) in every variant. Two small-regime corrections came out of the pre-commit review: the band must exceed the cycle width ($r > K$, plus one when balanced) for a burst’s replacement draw to exist, and the build now asserts this with a message instead of spinning; and an odd K rounds down to disjoint pairs ($K3 = K2$) in both mask kinds, so the balancing CNOT is emitted exactly when `balanced` is set. The exhaustive test now also covers $K = 3$, $K = 4$ and repair-kind slots.

4 Cost, and the `max_open 2` variant

The balancing CNOTs are a minor cost. The +92% comes from the **fire polynomial**: an operand under three open masks has 8 monomials in the plain build (the wire, the constant, and y , xy per mask) and 11 in the balanced one, so each fire expands to $11 \times 11 = 121$ monomials instead of $8 \times 8 = 64$, and there are $\approx 7,900$ fires at $n = 128$. In the packed deliverable all monomials of one fire target the same wire and fold into one packed gate, so the packed gate count grows much less than 92%, but the term count (evaluation time) grows with it.

`max_open` is the rolling cap on simultaneously open masks per wire and the read-time masking depth (top-ups to that count). At **`max_open 2`** the balanced read polynomial is back to 8 monomials per operand, 64 per fire:

build ($n = 128$)	gates	vs plain <code>max_open 3</code>
plain quad-fire, <code>max_open 3</code>	514,502	baseline
balanced, <code>max_open 3</code> (default)	984,384	+92%
balanced, <code>max_open 2</code>	565,374	+10%
plain quad-fire, <code>max_open 2</code>	307,486	−40%

`fire_corr` cannot tell balanced `max_open 2` from 3 (interior median 0.076 vs 0.078). The gauntlet can (§6): with a single open balanced mask $x \oplus z \oplus q$, the XOR of the wire’s segment with any visible monomial that contains z cancels the uniform term and leaves the biased quadratic part (phi 0.23); the two-feature scan finds such pairs when its strided subset holds one (6 of 1280 targets

at $n = 256$, $k = 256$) and the harness mixer wrecks the variant (a1 10, xrows 80, w1 201 mixed). Three open masks carry more independent uniform terms than a pair of features can cancel. **Both settings are kept** (BV5_MAX_OPEN, -bv5-max-open); **3 is the default**.

5 Encoded I/O and the gauntlet arm

The gauntlet traces every wire state, gate flip and new target value of a gadgetized execution and asks whether an observer can recover or correlate with the source’s intermediate values. To run the module in it, `gadgetize_blinded_v5` gained an **encoded I/O** mode (`BlindedV5Params::encoded_io`, off in production): `max_open` LGIs per data wire are opened *before the first gate* and the final drain is skipped; the gates that would open them are returned as `pre_gates` (apply to the plaintext state to encode) and the closing gates as `post_gates` (apply to the final state to decode). Neither is part of the traced circuit, so no raw input or plain output is ever a wire value the observer sees — the same convention as the share-encoded file arms. The band starts uniformly random and is not seeded from the inputs (at eight wires an input-seeded band has 256 contexts and every seed gate’s flip is a raw function of the input).

Harness changes (`tests/gauntlet/`): native arms `bv5` and `bv5bal` in `gauntlet_gen.rs` (production preset, `-bv5-band = max(32, n)`, `-bv5-max-open`; they refuse `-aux zero`, since a zero band makes every pre-opened mask a constant, and mark no target as a trivial raw-input feature, since no initial wire equals a raw input), orchestrator arms `blindedv5`, `blindedv5_balanced`, `blindedv5_balanced_mo2` and a `-n-wires` option for the native arms, auditor whitelist, README and pipeline notes. Generation is seconds even at $n = 256$; the prefix-affine audit pass dominates (~ 70 min for $n = 256$, $k = 512$ on the mix server, which has no NumPy, so only the `gen` and `audit` stages run there).

6 Gauntlet results

Counts are targets hit or flagged; five targets per source gate (a , b , c_{old} , f , c_{new}). **xtrace is omitted**: it is the full-trace affine identity for every XOR-masking arm including the colleague’s `none`, `secretshare14` and `bandproduct92` controls — each mask term is the flip of the LGI gate that applied it, so every plaintext is an exact XOR of trace features; only the colleague’s nonlinear-share arm passes it. Whether that relation is exploitable depends on identifying the mask gates without the targets, which is the segment-deduction line of attack, not this test.

Unmixed, balanced (default build), encoded I/O, random band:

n	k	a1	xrows	w1	w2	w3	targets
32	64	0	0	0	0	0	320
32	256	0	0	0	0	0	1280
64	64	0	0	0	0	0	320
64	256	0	0	0	0	0	1280
256	256	0	0	0	0	0	1280
256	512	0	0	0	0	0	2560

The plain quad-fire arm, for reference, unmixed: w1 flags every target (320/320, 1280/1280) and w2/w3 a few dozen — the one-mask channel of §2. The **none** control: a1 and xrows on most targets, w1 on all, w2/w3 on most.

max_open 2 balanced, unmixed, $n = 256$: $k = 256$ w1 4 $\rightarrow$ 0 before $\rightarrow$ after the burst-control rule, w2 6; $k = 512$ w2 20 $\rightarrow$ 0. Mixed (harness mixer, $k = 256$): a1 10, xrows 80, w1 201 — see §4.

Mixed cells use the harness’s own store-free mixer (crossings, copy splits, conjugation twists, 20k moves), not the pipeline. The balanced build shows sporadic w1 flags there (28 and 0 of 320 at $k = 64$; 47 and 60 of 1280 at $k = 256$; one xrows hit), the plain build keeps its w1 failures and gains exact a1 copies. The mechanism is the mixer’s own: a conjugation by a CNOT from a balancing wire onto a masked data wire removes that wire’s uniform term inside the conjugated segment, and copy splits leave partial mask terms between pieces. Production runs twists off, but store splices can create the same partial states inside a window, so the pipeline must be measured with **fire_corr** on real outputs rather than inferred from these cells.

7 What is not covered by this change set

- The pipeline-level measurement of the balanced build (phase A, 2-eff, packed final) with **fire_corr** and the affine plates, and the re-run of the $K = 2$ arms, whose earlier deliverables carry the idle-bare defect. The mix server’s generator binary must be redeployed from this commit first.
- The gauntlet’s w2/w3 scans are capped (64 and 16 features, strided); their zero is bounded evidence, as the colleague’s handbook states. w1 is exhaustive over features.
- The **max_open 2** variant’s residual against a targeted (not strided) two-feature attacker.

8 Files in this change set

file	change
src/preprocessing/blinded_v5.rs	balanced (default on), encoded_io, burst_band_only; six coverage rules; pre_gates/post_gates; diag counters; exhaustive test extended
src/preprocessing/bin/gen_sandwich_gadget.rs, blinded_v5_gadgetize.rs	env knobs BV5_BALANCED (both), BV5_BAL_SEED and BV5_BURST_BANDONLY (sandwich driver; max_open was already BV5_MAX_OPEN / positional); balanced band seed
scripts/gss_mix.sh	-bv5-max-open N, -bv5-balanced 0 1, -bv5-min-open N
red_team_tests/bin/leakage/fire_corr	new tools
(+ Cargo.toml bin),	
red_team_tests/bare_census.py,	
red_team_tests/open_mask_profile.py	
tests/gauntlet/	blinded-V5 arms, -n-wires, docs
gauntlet_gen.rs, gauntlet.py,	
gauntlet_audit.rs, README.md,	
TESTING_PIPELINES.md	
docs/BLINDED_V5_LGI_DESIGN.*	§2 rules, §4 balanced row, §5.6–5.7 measurements, §6, §7
src/README.md, docs/GSS_MIX.md	compute-stage description and knobs